Lab 01 — Internship, Discovery and Planner

Thirty minutes describing the problem. The rest of the session building the answer.

The Goal

You're going after summer internships, and keeping track of it all gets overwhelming fast — which ones you care about, whether you've applied, who you've spoken to, what you owe them, and what you'd have to change in your resume.

You want one place that holds all of it.

Step 01: Brainstorming and Discovery

About 30 minutes, as a guide rather than a limit.

Work as a team to discuss the questions below. Feel free to add some of your own — these are just starters.

Remember that we are building and iterating. As a group, use your judgment for when you have done enough discovery to move on to building.

Capture your answers on your template, whiteboard and sticky notes. You'll photograph them and hand them to Claude at the end.

- **Where Do You Normally Find Out About Internships?**
    - Public job sites, the Yale career site, a professor, a friend who already works there?
- **What Frustrates You Most About Applying?**
    - Be specific. The last time it went badly, what actually went wrong?
- **What Do You Wish You Could Remember About Each One — Before You Apply, and After?**
    - Deadlines, links, who you emailed, where you stand, what they said.
- **Are You Updating Your Resume / Cover Letter for Each Application? How?**
    - If you tailor them, what do you change, and how long does it take?
- **What Do You Find Out Too Late?**

    - What are the things you want to be notified about, and when?
- **What Should It Look Like When You Open It?**
    - A list, a calendar, a dashboard of what's urgent, something else? Sketch it.

Look Back at What You Just Wrote. That's Your Portal.

You didn't design software just yet. You described your own situation, and those are your requirements. What you build should flow naturally from them. Then you iterate, improving what you've built and adjusting it as conditions change.

Step 02: Hand It to Claude

- **Open a Cowork session** connected to the project folder you made in Lab 00.
- Send Claude your answers along with the prompt below. It asks for a *minimum viable product*: the simplest version that actually works, which you'll improve from there.

PROMPT

```
We're a table of Yale students building an internship planner for ourselves. It should help us keep track of the internships we're applying to, remember deadlines and contacts, and stop finding things out too late. We'll build it step by step: a simple working version first, then we'll improve it.

Here are our notes on how we track internship applications, and what goes wrong.

[attach your photos, and add any other notes from your brainstorming]

Build us an initial minimum viable product that we can further customize working with you.

Build it with:

- Streamlit, in Python. Please don't use a different framework.
- uv to install whatever it needs.
- One start file, so we can run it again later with a single command.
- A button that loads sample data, so we can see it working right away.
```

```
Before you build anything: ask us up to five questions about whatever is unclear, then
show us a mockup using sample data. Wait for us to approve the mockup before you write
the real thing.
```

> **Hint:** Claude handles images and photos as input very well. Photograph your template, whiteboard and sticky notes with a phone and attach the pictures — handwriting, arrows, crossings-out and all. You don't need to type any of it out.

Step 03: Answer Its Questions, Then Judge the Mockup

- Claude will ask you a handful of questions. **Answer them as a table.**
- Then it shows you a mockup: a rough version with made-up data, before any real code exists.
- **Look at it properly.** Is this the thing you described? What's missing? What's there that nobody asked for?
- Say what's wrong and ask for another one. **Do not approve a mockup you don't like** — changing it now costs seconds, changing it after the build costs ten minutes.

Step 04: Build It

- When the mockup is right, tell Claude to go ahead.
- Claude asks permission before it does things. Click **Allow**, or **Allow for this task** to reduce the clicking.

Step 05: Take a Break

Builds take **ten minutes or more**, so this is a good time for a break.

- **Keep someone close.** Claude pauses to ask questions and waits until somebody answers. Nothing moves while it waits.
- **When you're back, talk about what comes next:**
    - What would you add or change first?

    - Who at your table would actually use this next week, and what would change the others' minds?

- **Compare with your neighbors.** The table next to you made different choices from the same questions.

Step 06: Run It, Test It and Improve on It

- Claude will tell you how to start your app and give you a web address to open.
- **Don't see a change you asked for?** When Claude says it's done, check that it restarted the portal (ask it to if it didn't), then refresh your browser page. A refresh on its own isn't always enough.
- **Add a few internships you'd actually go for.** Find the posting and paste the link.
- **What would you like to improve?** Did it do everything you asked for, or did it leave some things out?
- **What did you learn about this process that you'd like to keep?** Ask Claude to write those best practices and design principles into a Markdown file in your project folder. Congratulations, you have your first skills file!
- Next time you build something, ask Claude to read the skills file as part of the prompt.
- **Step back and look at what you've built.** Does it look and work the way your table wants, or like a default? What else would you like in it: a different layout, a calendar view, sorting, another table? Ask Claude for it, then test it.

Step 07: Make It Do Something

So far, your portal shows you things. Now it's time to make it perform some actions. One thing you may want to add is reminders (you may already have this under "What Do You Find Out Too Late?").

Remember MCP, the "USB-C port" for AI? We built an MCP server for this class, so your portal has a tool it can use to take actions. One of its tools, `send_reminder`, emails a reminder, with a calendar invite if you give it a date, to your table's lab account, which forwards to your table leader's inbox. It can only send to the lab accounts, which is generally good practice: it keeps personal email addresses masked. The server lives at `https://mcp.paradigmventures.ai/mcp`.

See how it fits together.

Suggestion: Add a "Send Reminder" Button

- **Ask Claude to add the reminder button.** Start from this, and change it to fit what your table wants:

> **PROMPT**
>
> Add a "Send Reminder" button to our planner. When we click it, the portal's own code should connect to the workshop MCP server at https://mcp.paradigmventures.ai/mcp and use its send_reminder tool to send a reminder email or calendar invite about that internship to our table's lab account.
>
> - Our table's lab account is demo-XX@paradigmventures.ai.
> - The server's access key is: `LCpM2l-L1_Vd1wybj5KPHovFDYNmKq5y`. Keep it in a local settings file, not in the code.
> - Use the current official MCP Python library (`mcp`), and check how its current version works rather than relying on memory.
> - Don't send real reminders while you test. We'll send the first one ourselves from the portal.

- **Check that it worked.** Make sure Claude restarted the portal (if it doesn't tell you, ask it how). Then refresh your browser page. Verify that the reminder button is there. Click the reminder button, then open the table leader's inbox. Did the message arrive, and is it what you asked for?

Suggestion: Add a Chat Window

Adding a chat window lets your table type a request in plain English, using a small AI model running on the table leader's laptop. Now the portal becomes a place where you can start requesting increasingly sophisticated tasks.

- **Ask Claude to add the chat window.** Start from this:

> **PROMPT**
>
> Add a small chat window to our planner, using the local Ollama model that's already installed. When we type a request such as "remind us about Goldman on Friday," send it to the model and have it fill in a fixed format, using Ollama's structured outputs: the action, the internship and the date. The portal's own code should then check that answer and carry it out, using the workshop server's reminder tool. If the answer doesn't fit the format, show us a message and don't send anything.
>
> - Use the qwen3.5:4b model, with its thinking turned off (think: false), so replies come back quickly.
> - Use the same workshop server address, lab account and access key as above, or ask us for them if we haven't given them to you yet.

- Don't install Ollama or download the model in your own workspace to test it. You can't reach the one on our laptop from there. Write the code, test the checking with a few sample answers, and we'll try it with the real model when we run the portal.
- Make the chat window easy to find: put it near the top of the sidebar, with a clear heading and a text box a few lines tall.
- Don't send real reminders while you test. We'll send the first one ourselves from the portal.
- Keep it simple: a first version we can try within a few minutes. We'll tell you what to improve.

The model, `qwen3.5:4b`, is the one your table leader installed in Lab 00.

- **Restart and refresh.** As with the button, make sure Claude restarted the portal (if it doesn't tell you, ask it how), then refresh your browser page.
- **Try it, then try to break it.** Ask for something reasonable, then something strange. What does it do when it doesn't understand you?
- **Look under the hood.** Before the model reads your message, your portal gives it instructions of its own: a system prompt. What do you think it says? Ask Claude to show you:

PROMPT

```
Add a "System Prompt" page to the sidebar that shows the exact instructions the portal sends to the model with each message, and lets us edit and save them, with a button to restore the original.
```

- **Change one rule and see what happens.** It's the same idea as the skills file you wrote in Step 06.
- **Notice what you just built.** You type a request, and the system takes an action for you. A small model can handle a simple task like this without going to the cloud.

Step 08: Make It Yours

Whatever time is left belongs to you. Go back to your brainstorming sheet and look at your portal honestly:

- **Did it hit everything?** What's on your sheet that isn't in the portal?
- **Is it easy to use?** Could someone at your table use it without help?
- **Would you really use it?** If not, what would change your mind?

Choose what matters most, ask Claude for it in your own words, then test it.

If You Get Stuck

- **Strange file errors, or the build keeps failing:** check your project folder isn't in iCloud, Dropbox, OneDrive, Box or Google Drive. Move it to your home folder and try again.
- **Claude seems frozen:** look for a permission prompt waiting for a click.
- **The app won't start:** paste the error message back to Claude and ask it to fix it. This is normal and expected — it is not a sign you did something wrong.
- **You're out of ideas:** see the Answer Sheet at the bottom.

Key Takeaways

- **The thirty minutes of talking did more for your build than any single prompt.** You described a problem; the design came out of the description.
- **You reviewed a plan before accepting the work.** That one habit separates people who get good results from people who get impressive-looking ones that break.
- **You directed an agent.** It asked, planned, used tools and built — you decided what mattered.
- **A thing that displays is a page. A thing that acts is an agent.** "What Do You Find Out Too Late?" is where yours crossed that line.

Answer Sheet

Use these if your table is stuck. Reading them first spoils the exercise.

[PLACEHOLDER - suggested prompts: a worked starting prompt, a prompt for adding the agentic action, and two recovery prompts for common failures]

What's Next

Your app lives in your project folder. Share it with your table, keep building, or start something else entirely — you now know how.